

ENEE452 HW07 : critical region analysis; FreeRTOS Blinky

This homework will look at a critical region problem and create a Blinky with FreeRTOS

HW07starter_ws is the early development of a messaging protocol developed specifically for having 'conversations' with smart devices. The goal is to be able to leverage something like the 'printf()' function for use in exchanging complex commands and data with smart devices like those on the B-L475_IOT1A board. To that end a toy problem which sends text messages to the serial console was solved. Code development was based on HW03start_ws so that CubeMX, HAL, and LL were available for support.

The result of the development in HW07starter_ws is a working demo of two functions:

- msgPost() -- a fancy sprintf()-like function which places its result in a communication queue;

- msgSend() -- a function called by the communication queue manager to send messages from the queue.

A major simplification of the real problem was made to speed development: no interrupts were used in the messaging operations. This was done so work could focus on operation of the data structures, but is unrealistic for actual operation as the communication links its designed for (serial, SPI, I2C) are almost always interrupt-driven.

Problem 1) (4 points) Identify critical regions in msgPost() and msgSend()

The developments of msgPost() and msgSend() in consolemsg05.c are safe from critical region issues because they both run in the foreground and are called in alternation in main(). However, for actual use the msgSend() queue manager is almost certainly going to be called at random intervals triggered by a communication port's interrupt. On the other side of things, processes controlling the board's fancy peripherals will be doing so by posting messages to them at random moments determined by an RTOS's assignments of timeslices.

The result of the above is a guarantee of collisions between msgPost() and msgSend() in any critical regions which exist in the design.

For this problem: identify by line numbers the minimal (no larger than necessary) critical regions which exist in operation of the code in consolemsg05.c. Assume that msgPost() and msgSend() are both running in interrupts. For each critical region you identify, describe the sequence of events for which a collision would give a bad result.

Problem 2) (4 points) BlinkyRTOS_ws

Modify HW07starter_ws so that it runs under FreeRTOS.

Give it two tasks:

- LEDcontroller() -- determines LED 'turn on' and 'turn off' times, communicates ON/OFF to LEDperipheral().

- LEDperipheral() -- switches LED ON/OFF based on information from LEDcontroller().

This level of the problem is just to get you started with freeRTOS so you will probably be best served using something like a semaphore to communicate between the two tasks

WHAT TO SUBMIT – two project ‘_ws’ zipfiles contained in the Homework zipfile submission.

For another homework: Problem 3) msgBlinkyRTOS_ws

- Replace the signalling method you used for Problem 2's control over the LED with messages to be posted by LEDcontroller() to a list managed by LEDperipheral().*
- Create a separate message list to be managed by the serial console interrupt.*
- Have LEDcontroller() send messages "I'm turning LEDperipheral ON" and "I'm turning LEDperipheral OFF" to the serial console's list at the appropriate times, and have LEDperipheral() send messages "I'm turning ON now" and "I'm turning OFF now" to the serial console list when appropriate.*